University of Tehran
ECE

Social Network

Instructor: Masoud Asadpour

Teacher Assistant:
Amirhossein Roshandel
Amin Motamedi

Homework 4
Winter 2026

Submission Guidelines and Policies

Submit your final file as a single **ZIP** file that includes the report in **PDF** format and all code files, and upload it to the **eLearn platform**. The name of the ZIP file should follow the pattern: `SN_HW#_StudentNumber`

- If you have any questions, contact the assignment TAs via **email**. Please avoid sending private messages on social media so that responses can remain organized and efficient.
- The length of the report is not a grading factor. For implementation questions, focus on providing clear explanations; clarity matters far more than word count.
- The procedure for submitting assignments is explained in detail separately in this file and in the Git workshop video. Before submitting your code, ensure that you have watched the entire workshop video.
- Every submission must include both the report and the corresponding code. Any code submitted without a report will receive **zero points**.
- In your assignment report, you must describe how you used these tools. Include details such as the tools you used, their specific applications, and any other relevant information.
- At the end of your report, you must include the link to the prompts used: (`The ChatGPT conversation link`).
- Assignments may be uploaded to eLearn for up to 7 days after the official deadline, but a **5% penalty per day** will be deducted from the grade for each late day. After 7 days, submissions will not be accepted.
- To verify your understanding of each assignment, there will be a brief 5–10 minute in-person or virtual review session. You will be selected for this session **once during the semester**. If there are discrepancies between your submitted report and your presentation, the chance of being chosen again will increase.
- **Plagiarism is strictly prohibited.** Any similarity in the report or code that indicates copying or if cheating occurs during exams, will result in a score of **0.25 for all students involved for the course.**

Contents

Cold War Intelligence Brief - 1985

Year 1985, the height of the Cold War. You are a CIA analyst who has recently obtained classified documents from the KGB. These documents contain suspicious communications between various individuals across Eastern Europe.

The data is incomplete and noisy. Some connections might be fake, some individuals might belong to multiple cells, and you must use theoretical analysis and multiple community detection algorithms to uncover the truth.

Your Mission Objectives:

- Identify different spy cells (communities) in the network
- Find key individuals who bridge between cells
- Discover the hierarchical structure and power dynamics
- Assess the quality of cell partitions using modularity

Theoretical Questions

Question 1: Identifying Cell Structures (15 points)

Intelligence Report: The following network shows communications between 12 suspected spies. Initial analysis suggests three potential cells (A, B, C) with two bridge connections between them.

Network Configuration:
Graph G with 12 nodes and the following edges:

- **Cell A:** (1,2), (1,3), (1,4), (2,3), (2,4), (3,4)
- **Cell B:** (5,6), (5,7), (6,7), (6,8), (7,8)
- **Cell C:** (9,10), (9,11), (10,11), (10,12), (11,12)
- **Inter-cell bridges:** (4,5), (8,9)

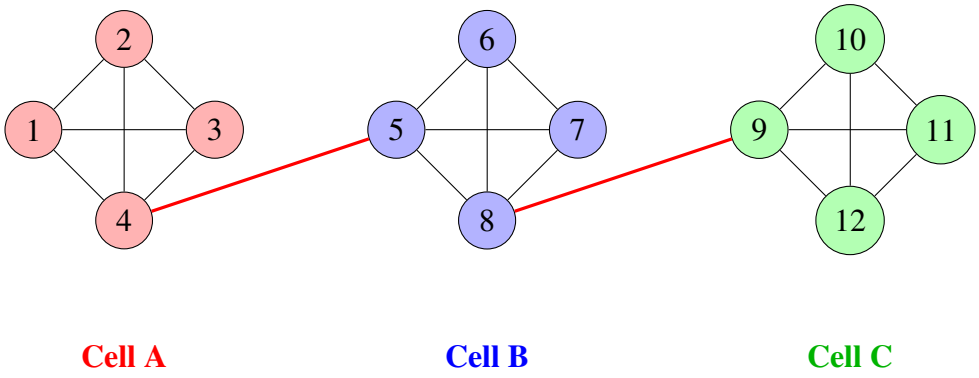

Figure 1: Spy Network Structure (Red edges = bridges between cells)

(a) Clique Analysis (5 points)

Operational Questions:

i. Find all cliques of size 3 or larger in the network

ii. Which cell is more cohesive (i.e., closer to a complete graph)? Explain why this matters for operational security.

iii. If edge (2,4) is removed (e.g., agent arrested), which cliques disappear? What does this tell us about network vulnerability?

(b) N-Clique and N-Clan Analysis (5 points)

Communication Analysis: Spies can exchange encrypted messages either directly or through one trusted intermediary.

i. Find all 2-cliques in this graph (subgraphs where any two nodes are at distance ≤ 2 in the full graph)

ii. Which of these 2-cliques are also 2-clans? (A 2-clan is a 2-clique with diameter ≤ 2 within the subgraph itself)

iii. **Strategic Question:** For modeling actual spy cells where agents use cutouts (intermediaries), which definition is more realistic: clique or 2-clique? Provide operational reasoning.

(c) K-Core Analysis (5 points)

Core Structure Identification:

i. Identify the 2-core and 3-core of this graph

ii. In which core(s) is spy number 4 located? What does this tell us about their centrality?

iii. **Methodological Question:** For finding the ”central core” of a spy network, which concept is more practical: clique or k-core? Justify your answer considering computational complexity and network properties.

Question 2: Power and Control in Networks (15 points)

Power Analysis Brief: Not all spies are equally important. Your task is to identify which individuals hold strategic positions that give them disproportionate influence over information flow.

Continue analyzing the same network from Question 1.

(a) Structural Holes and Local Bridges (6 points)

Information Brokerage Analysis:

i. Calculate the **embeddedness** of each edge, where embeddedness(u, v) = number of common neighbors of u and v

ii. Identify all **local bridges** (edges with embeddedness = 0)

iii. **Power Index Design:** Spies 4 and 8 both sit on local bridges. Design a "Power Index" to determine which one is more powerful. Your index should incorporate:

- Number of local bridges the individual controls
- Embeddedness of those bridges (lower embeddedness = more power)
- The spy's degree within their own cell

Calculate the Power Index for both spies 4 and 8, and determine who is more strategically important.

(b) Burt's Structural Holes Theory (4 points)

Theoretical Foundation:

i. According to Ronald Burt's theory of structural holes, explain why individuals positioned on structural holes (gaps between clusters) have more power than those embedded in dense clusters. What are the mechanisms of this power?

ii. In our spy network, who has the greatest access to structural holes? How does this translate to operational advantage?

iii. **Strategic Comparison:** Who is more powerful in an intelligence network:

- A spy with high degree (many connections) within a dense, cohesive cell, OR
- A spy with lower degree but positioned on a bridge between two cells?

Justify your answer using network theory concepts.

(c) Weak Ties vs. Strong Ties (5 points)

Information Diffusion Analysis:
Define ties as follows:

- **Strong tie** = edge within a cell (e.g., 1–2 in Cell A)
- **Weak tie** = edge between cells (e.g., 4–5 between A and B)

i. According to Granovetter's "Strength of Weak Ties" theory, which type of tie is more important for spreading *new* information across the network? Why?

ii. **Disinformation Campaign:** If the KGB wants to spread false intelligence throughout the entire CIA network, which specific edges should they target? Explain your strategy.

iii. **The Paradox:** Explain why weak ties—which are "weak" in terms of frequency and intimacy—are actually *stronger* for the overall network structure and information flow. What would happen to the network if all weak ties were removed?

Question 3: Modularity and Partition Quality (15 points)

Cell Quality Assessment: Your preliminary analysis has partitioned a 9-node network into three suspected cells. You must now quantitatively assess whether this partition truly represents distinct operational units.

Network Configuration

Total: 9 nodes, 13 edges
Partition:

- Community A: $\{1, 2, 3\}$
- Community B: $\{4, 5, 6\}$
- Community C: $\{7, 8, 9\}$

Internal Edges:

- Within A: (1,2), (1,3), (2,3) $\rightarrow$ 3 edges
- Within B: (4,5), (4,6), (5,6) $\rightarrow$ 3 edges
- Within C: (7,8), (7,9), (8,9) $\rightarrow$ 3 edges

Inter-community edges:

- A–B: (3,4)
- B–C: (6,7), (5,7)
- A–C: (2,8)

(a) Calculate Initial Modularity (6 points)

Use Newman's modularity formula:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(C_i, C_j) \tag{1}$$

where:

- m = total number of edges
- A_{ij} = 1 if edge exists between i and j, 0 otherwise
- k_i = degree of node i
- $\delta(C_i, C_j)$ = 1 if i and j are in same community, 0 otherwise

Required Analysis:

i. Create a table showing the degree k_i for each node (1 through 9)

ii. Calculate the modularity Q for this partition. Show your work step-by-step.

iii. Interpret the result: Is this Q value good or bad? (Recall: Q ranges from approximately -0.5 to $+1.0$, with $Q > 0.3$ generally indicating good community structure)

(b) Improving the Partition (5 points)

Reclassification Scenario: Intelligence suggests agent 3 might actually belong to a different cell.
Consider two options:

- **Option 1:** Move node 3 from Community A to Community B
- **Option 2:** Move node 3 from Community A to Community C

Required Calculations:

i. For each option, calculate ΔQ (the change in modularity) using the local modularity change formula:

$$\Delta Q = \left[\frac{\Sigma_{in} + k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right] \tag{2}$$

where:

- Σ_{in} = sum of edge weights inside the community (before move)
- Σ_{tot} = sum of degrees of nodes in community (before move)
- k_i = degree of node being moved
- $k_{i,in}$ = sum of edge weights from i to nodes in target community

ii. Which move results in better modularity? Should we reclassify agent 3?

(c) Resolution Limit (4 points)

Theoretical Limitation Analysis:

i. Explain the concept of **resolution limit** in modularity optimization. What does it mean mathematically and operationally?

ii. Why can't modularity optimization detect very small communities (e.g., 3–4 nodes) in large networks? Relate this to the modularity formula structure.

iii. **Applied Scenario:** Suppose our spy network contains a "dormant cell" of just 2 agents who maintain minimal contact with the main network (connected through only a single link to one person). Will modularity optimization successfully identify this as a separate community? Why or why not?

What alternative approaches might detect such small, isolated cells?

Implementation Questions

Question 4: Operation "Chain Breaker" - Girvan-Newman Algorithm (30 points)

Mission Briefing: You have successfully infiltrated the enemy communication network. Your next objective is to systematically dismantle it by identifying and severing the most critical communication channels. This operation uses the Girvan-Newman algorithm to methodically break down the network.

Target Network: Zachary's Karate Club - a real-world social network that historically split into two factions. For this operation, treat these factions as two competing spy cells.

Operational Objectives

1. Implement edge removal based on betweenness centrality
2. Track network fragmentation after each removal
3. Calculate modularity at each step to identify optimal partition
4. Compare detected cells with ground truth intelligence
5. Identify the critical communication link that splits the network

(a) Algorithm Implementation (15 points)

Implementation Task:
You are required to implement the Girvan-Newman algorithm in Python. A template file has been provided: `q4_girvan_newman.py`

Core Functions to Implement:

1. `girvan_newman_analysis(G, target_communities=2)`
   - Execute the iterative edge removal process
   - Track modularity and components at each step
   - Return comprehensive results dictionary
2. `calculate_modularity(G, communities)`
   - Compute Newman's modularity Q
   - Formula: $Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(C_i, C_j)$
3. `visualize_results(G, results, true_labels)`
   - Generate 5 intelligence report visualizations
   - Plot modularity progression
   - Create confusion matrix
4. `calculate_accuracy(detected_communities, true_labels)`
   - Compare detected partition with ground truth
   - Handle label alignment (labels may be reversed)

(b) Intelligence Analysis (8 points)

After executing the operation, provide detailed written analysis answering the following:
1. Modularity Progression Analysis (3 points):

- At which step did modularity reach its maximum value?
- Is maximum modularity achieved when the network first splits into 2 components?
- Why does modularity eventually decrease after reaching the optimum?

2. Critical Edge Analysis (3 points):

- Which edge was the first to fragment the network into two components?
- What special characteristics does this "critical link" possess?
  - Degree of connected nodes
  - Edge betweenness value
  - Position in network structure
- Was this edge actually connecting the two real factions?

3. Ground Truth Comparison (2 points):

- What accuracy was achieved compared to known faction membership?
- If not 100%, which agents were misclassified?
- Analyze why these specific agents were difficult to classify (examine their local network structure)

(c) Strategic Report (7 points)

Prepare a one-page operational assessment including:
1. Operation Summary (2 points):

- Total number of communication links severed before network fragmentation
- Identification of the critical communication channel

2. Scalability Analysis (3 points):

- Time complexity: $O(m^2n)$ where m = edges, n = nodes
- Is this algorithm viable for large-scale networks (¿1000 nodes)?
- Computational bottlenecks and limitations

3. Operational Recommendation (2 points):

- Would you recommend this method for rapid intelligence operations?
- What are the trade-offs between accuracy and speed?
- Suggest alternative approaches for time-critical scenarios (e.g., Louvain, Label Propagation)

Question 5: Operation "Rapid Detection" - Algorithm Comparison (25 points)

New Intelligence: Time is critical. A larger network has been discovered—the communication patterns from Victor Hugo's *Les Misérables*. You must deploy multiple detection algorithms simultaneously and compare their effectiveness.

Multi-Algorithm Deployment

Target Network: Les Misérables character interaction network

- 77 nodes (characters)
- 254 edges (co-appearances in scenes)
- Unknown number of communities

Deployment Strategy: Execute three detection algorithms in parallel:

1. **Louvain** - Fast modularity optimization ($O(n \log n)$)
2. **Label Propagation** - Ultra-fast but non-deterministic ($O(m + n)$)
3. **Greedy Modularity** - Baseline method ($O(mn)$)

(a) Multi-Algorithm Implementation (12 points)

Implementation Task:
You must implement three community detection algorithms and compare their performance.
Template file: `q5_algorithm_comparison.py`
Functions to Implement:

1. `algorithm_comparison(G, num_runs=5)`
    - Execute all three algorithms multiple times
    - Record execution time, modularity, and community statistics
    - Return pandas DataFrame with comprehensive comparison
2. `label_propagation(G, max_iter=100, seed=None)` **[Implement from scratch!]**
    - Initialize each node with unique label
    - Iteratively update labels based on neighbor majority
    - Handle ties with random selection
    - Return labels and iteration count
3. `deep_analysis(G, best_communities)`
    - Identify bridge characters (connected to 3+ communities)
    - Find most central person in each community
    - Calculate inter/intra-community edge ratios
    - Generate community size distribution

Required Metrics for Comparison:

- Average execution time (seconds)
- Standard deviation of time
- Average number of communities detected
- Average modularity score
- Standard deviation of modularity (measures determinism)
- Largest and smallest community sizes

Libraries You May Use:

- `community_louvain.best_partition(G)` for Louvain
- `nx.algorithms.community.greedy_modularity_communities(G)` for Greedy
- Label Propagation must be implemented from scratch (no library allowed)

(b) Comparative Analysis (7 points)

Generate and analyze the algorithm comparison table. Your table should look similar to:

Algorithm	Time(s)	#Comm	Modularity	Largest	Smallest
Louvain	?	?	?	?	?
Label Propagation	?	?	?	?	?
Greedy Modularity	?	?	?	?	?

Table 1: Algorithm Performance Comparison

Required Written Analysis:
1. Speed Analysis (2 points):

- Which algorithm executed fastest?
- Explain why based on computational complexity
- Does empirical runtime match theoretical complexity?

2. Quality Analysis (2 points):

- Which algorithm achieved highest modularity?
- Is there a clear speed-quality trade-off?
- Do all algorithms find similar number of communities?

3. Determinism Analysis (2 points):

- Compare standard deviation of modularity across algorithms
- Why does Label Propagation show higher variance?
- Is non-determinism a bug or a feature? Discuss advantages and disadvantages

4. Recommendation (1 point):

- Which algorithm would you deploy for operational use?
- When should each algorithm be used? (Consider: speed requirements, accuracy needs, network size)

(c) Network Intelligence Report (6 points)

Analyze the Les Misérables network structure using the best algorithm (highest modularity):
1. Bridge Character Analysis (2 points):

- List all characters connected to 3 or more communities
- Research their roles in the novel (use external sources)
- Explain whether their narrative role justifies their bridge position

- Example: Does Jean Valjean connect multiple communities? Why does this make sense?

2. Community Leaders (2 points):

- Identify the most central character in the largest community
- Calculate both degree centrality and betweenness centrality
- Does this align with the story's protagonist?
- What does this tell us about narrative structure vs. network structure?

3. Network Disruption Strategy (2 points):

- If the goal is to maximally fragment the network, which characters should be targeted for removal?
- Justify your targets using network metrics (high betweenness, bridge positions)
- Simulate: What happens if all bridge characters are removed? How many components result?

Bonus Question 6: Operation "Dormant Cell" Detection (20 points)

Classified: Dormant Cell Intelligence

Threat Assessment: Intelligence suggests the existence of "dormant cells"—small, tightly-knit groups that intentionally minimize external contact to avoid detection. Standard community detection algorithms may miss these cells due to the resolution limit of modularity optimization.
Your Mission: Design and implement a custom detection algorithm specifically for identifying dormant cells based on suspicious behavioral patterns.

(a) Suspicion Score Design (10 points)

Theoretical Framework:
Design a mathematical formula to quantify how "suspicious" each detected community appears. Your formula must incorporate four key indicators:
1. Internal Density (Weight: $w_1 = 0.35$):

$$\text{density}_{internal} = \frac{\text{\# actual edges within community}}{\text{\# maximum possible edges}} \tag{3}$$

Rationale: Dormant cells maintain tight operational security through dense internal communication networks.
2. External Isolation (Weight: $w_2 = 0.35$):

$$\text{connectivity}_{external} = \frac{\text{\# edges to outside communities}}{\text{\# total possible external edges}} \tag{4}$$

Rationale: Compartmentalization principle—minimal contact with outsiders reduces exposure risk.

3. Size Optimality (Weight: $w_3 = 0.15$):

$$\text{size\_score} = 1 - \frac{|\text{community size} - \text{average size}|}{\text{average size}} \tag{5}$$

Rationale: Operational cells are neither too large (conspicuous) nor too small (ineffective). Prefer medium-sized groups.

4. Network Peripherality (Weight: $w_4 = 0.15$):

$$\text{peripherality} = 1 - \text{average closeness centrality of members} \tag{6}$$

Rationale: Dormant cells operate at network periphery to avoid surveillance, not in central, high-visibility positions.

Combined Suspicion Score:

$$\boxed{\text{Suspicion} = w_1 \cdot \text{density}_{internal} + w_2 \cdot (1 - \text{connectivity}_{external}) + w_3 \cdot \text{size\_score} + w_4 \cdot \text{peripherality}} \tag{7}$$

Deliverables:

1. Justify the choice of weights (w_1, w_2, w_3, w_4). Why is internal density most important?
2. Explain why these four factors are critical for dormant cell identification
3. Propose at least one alternative formulation and discuss trade-offs
4. Discuss potential weaknesses: What types of cells might this formula miss?

(b) Detection Implementation (10 points)

Implementation Task:
Implement the dormant cell detection algorithm. Template file: `q6_dormant_cell.py`

Function to Implement:
`find_dormant_cell(G, partition)`

- Calculate suspicion score for every community in the partition
- Identify community with highest suspicion score
- Compute all four characteristic metrics
- Generate detailed reasoning for why this community is suspicious
- Return comprehensive results dictionary

Testing Requirements:

- Apply your detector to the Les Misérables network
- Use the best partition from Question 5

- Identify the most suspicious community
- Analyze which characters are flagged

Deliverables:

1. Working implementation: 6 points
    - Correctly calculates all four metrics
    - Handles edge cases (single-node communities, isolates)
    - Returns properly formatted results
2. Identification and analysis: 4 points
    - Which community in Les Misérables has highest suspicion score?
    - List all members of this ”dormant cell”
    - Research: What are these characters’ roles in the story?
    - Explain: Why does the network structure flag them as suspicious?
    - Does this make narrative sense, or is it a false positive?

Visualization Requirements:

- Plot 1: Full network with suspected dormant cell highlighted in red
- Plot 2: Subgraph showing only dormant cell members and their connections
- Plot 3: Bar chart comparing suspicion scores across all communities

Mission Complete

End of Implementation Assignment

Submission Requirements:

- All Python implementation files (`q4_*.py`, `q5_*.py`, `q6_*.py`)
- Generated visualizations (saved as PNG/PDF in `figures/` folder)
- Written analysis reports (PDF format)
- Results table (CSV or Excel format)

Good luck, Agent. The intelligence community is counting on you.

Classification: TOP SECRET